

2025 / 2026

DEVELOPPEMENT ET FRAMEWORK BACK-END



M. TEJIOGNI

Email : mtejiogni@yahoo.fr

Tél : 693 90 91 21

PLAN DU COURS

OBJECTIF GENERAL	3
OBJECTIFS SPECIFIQUES	3
CHAPITRE I : INTRODUCTION AU FRAMEWORK BACK-END	4
Objectif du chapitre.....	4
Objectifs spécifiques.....	4
I. Présentation des Frameworks Back-end.....	4
I.1. Définition des Frameworks Back-end	5
I.2. Architecture des Frameworks Back-end.....	5
I.3. Avantages des Frameworks Back-end.....	5
I.4. Inconvénients des Frameworks Back-end	6
I.5. Choix du Framework en Fonction des Besoins du Projet	6
II. Comparaison entre différents Frameworks	6
II.1. Symfony	6
II.2. Laravel.....	7
II.3. Django	7
II.4. Sprint Boot	8
II.5. Comparaison Globale	8
III. Installation et configuration de Python Django	9
Tests de connaissances.....	13

OBJECTIF GENERAL

L'objectif de ce cours est de fournir une compréhension approfondie des frameworks back-end dans le développement de logiciels, en mettant l'accent sur l'architecture, la gestion des données, et le développement des fonctionnalités. Les étudiants apprendront à choisir le framework le plus adapté en fonction des besoins d'un projet, qu'il s'agisse de Symfony, Laravel, Django ou Spring Boot. À travers une approche pratique, ce cours permettra aux participants de développer une application complète en utilisant les meilleures pratiques de l'industrie, en intégrant des concepts tels que l'architecture MVC, la gestion des dépendances, et la mise en œuvre des APIs. En fin de parcours, les étudiants seront en mesure de concevoir, développer, tester, et déployer des applications web robustes et performantes, tout en s'assurant d'une maîtrise des aspects de sécurité et de performance.

OBJECTIFS SPECIFIQUES

- **Comprendre les Fondamentaux des Frameworks Back-end** : Expliquer les concepts de base liés aux frameworks back-end, leur rôle dans le développement web, ainsi que les avantages et inconvénients associés à leur utilisation.
- **Comparer Divers Frameworks** : Analyser et comparer les principaux frameworks back-end (Symfony, Laravel, Django, Spring Boot) pour aider les étudiants à faire un choix éclairé en fonction des exigences spécifiques de leurs projets.
- **Installer et Configurer un Framework** : Proposer un guide détaillé sur l'installation et la configuration du framework choisi, en intégrant les outils et dépendances nécessaires pour démarrer un projet.
- **Appliquer l'Architecture MVC** : Démontrer comment organiser un projet en suivant l'architecture MVC, en mettant l'accent sur la séparation des responsabilités entre le modèle, la vue, et le contrôleur.
- **Gérer une Base de Données** : Démystifier la connexion à des bases de données, en utilisant des requêtes SQL ou NoSQL, tout en intégrant un ORM pour faciliter la gestion des données.
- **Développer des Fonctionnalités Complètes** : Créer des contrôleurs et des vues dynamiques, gérer les formulaires, les validations et la logique métier nécessaire au bon fonctionnement de l'application.
- **Mettre en Place la Sécurité** : Étudier les meilleures pratiques en matière de sécurité, y compris la gestion des sessions, la protection contre les attaques, et la gestion des exceptions.
- **Tester et Déployer l'Application** : Former les étudiants à tester leurs applications et à déployer efficacement dans un environnement de production, en intégrant des stratégies de monitoring et de gestion des logs.

CHAPITRE I : INTRODUCTION AU FRAMEWORK BACK-END

Objectif du chapitre

L'objectif de ce chapitre est de fournir une compréhension détaillée des frameworks back-end, en explorant leur rôle essentiel dans le développement d'applications web modernes. Les étudiants apprendront à évaluer les avantages et inconvénients de divers frameworks, à choisir celui qui correspond le mieux aux exigences de leurs projets, et à procéder à une installation et configuration efficaces.

Objectifs spécifiques

- **Définir les Frameworks Back-end** : Présenter une définition claire des frameworks back-end et expliquer leur fonction dans le cycle de vie du développement d'applications.
- **Analyser les Avantages et Inconvénients** : Identifier et discuter les principaux avantages et inconvénients des frameworks back-end, notamment en termes de rapidité de développement, de modularité, et de courbe d'apprentissage.
- **Évaluer les Besoins du Projet** : Développer des compétences pour évaluer les besoins spécifiques d'un projet afin de choisir le framework le plus adapté, en tenant compte des critères comme la scalabilité, la performance, et la communauté de support.
- **Comparer les Frameworks** : Réaliser une comparaison approfondie entre plusieurs frameworks populaires, tels que Symfony, Laravel, Django, et Spring Boot, en abordant leurs caractéristiques distinctes, leur écosystème, et leur utilisation typique.
- **Procéder à l'Installation et Configuration** : Fournir un guide pas-à-pas pour l'installation et la configuration du framework choisi, y compris l'installation des dépendances et les meilleures pratiques pour les paramètres initiaux.

I. Présentation des Frameworks Back-end

Dans le monde du développement web, les frameworks back-end jouent un rôle fondamental en facilitant la création d'applications robustes et évolutives. Ces frameworks sont des ensembles d'outils et de bibliothèques préconçus qui permettent aux développeurs d'écrire le code d'une manière plus structurée et efficace. En d'autres termes, un framework back-end fournit une base sur laquelle les développeurs peuvent construire des fonctionnalités complexes tout en évitant de reproduire des solutions déjà existantes.

I.1. Définition des Frameworks Back-end

Un framework back-end est une plateforme logicielle qui fournit les éléments fondamentaux nécessaires pour le développement de l'architecture d'une application. Contrairement au développement front-end, qui se concentre sur la présentation des données et l'interaction utilisateur, le back-end s'occupe de la logique métier, de la gestion de la base de données, et des communications entre le serveur et le client. Les frameworks back-end offrent les outils nécessaires pour gérer ces aspects sans avoir à partir de zéro.

I.2. Architecture des Frameworks Back-end

La plupart des frameworks back-end sont construits autour d'architectures éprouvées, telles que l'architecture Model-View-Controller (MVC). Dans cette architecture, la logique d'application est séparée en trois composants :

- **Modèle (Model)** : Gère les données et la logique métier.
- **Vue (View)** : Présente les données à l'utilisateur.
- **Contrôleur (Controller)** : Gère les entrées de l'utilisateur et interagit avec le modèle et la vue.

Cette séparation permet aux développeurs de travailler sur différentes parties de l'application sans interférer les uns avec les autres.

I.3. Avantages des Frameworks Back-end

Les frameworks back-end offrent de nombreux avantages :

- **Productivité Accrue** : En fournissant des fonctionnalités préconçues et des patterns de développement, les frameworks permettent aux développeurs de gagner du temps et de se concentrer sur les spécificités de leur application.
- **Sécurité** : La plupart des frameworks intègrent des fonctionnalités de sécurité dès le départ, comme la protection contre les injections SQL, les attaques par cross-site scripting (XSS), et d'autres vulnérabilités courantes.
- **Scalabilité** : Avec des architectures conçues pour gérer des charges croissantes, les frameworks back-end permettent aux applications de croître avec les utilisateurs et les données sans nécessiter une refonte complète.
- **Communauté de Support** : Un cadre bien établi aura souvent un large écosystème de développeurs qui contribuent à son amélioration continue, ainsi qu'une documentation riche pour aider les utilisateurs à résoudre les problèmes.
- **Modularité** : Les frameworks permettent d'organiser le code en modules, facilitant ainsi la réutilisation et la maintenance du code sur le long terme.

I.4. Inconvénients des Frameworks Back-end

Bien qu'ils offrent de nombreux bénéfices, les frameworks back-end peuvent également présenter des inconvénients :

- **Complexité** : Pour les petits projets, un framework peut parfois ajouter une complexité inutile en raison de sa structure et de ses exigences.
- **Courbe d'Apprentissage** : Chaque framework a sa propre syntaxe et ses conventions, ce qui peut représenter une barrière à l'entrée pour les nouveaux développeurs.
- **Dépendance à la Plateforme** : Lorsqu'un projet est construit sur un framework spécifique, il peut être difficile et coûteux de migrer vers un autre framework à l'avenir.
- **Performance** : Certains frameworks peuvent introduire une surcharge, ce qui peut nuire à la performance de l'application si elle n'est pas gérée correctement.

I.5. Choix du Framework en Fonction des Besoins du Projet

Le choix d'un framework dépend largement des besoins spécifiques du projet. Par exemple, pour une application qui nécessite des performances optimales, un framework léger comme Flask (en Python) ou Express (en Node.js) pourrait être approprié. D'autre part, pour une application d'entreprise complexe qui nécessite une grande modularité et un support de la communauté, des frameworks comme Symfony ou Spring Boot pourraient être préférables.

II. Comparaison entre différents Frameworks

La sélection d'un framework back-end est une étape cruciale pour tout projet de développement d'applications web. Ce choix influencera non seulement le processus de développement, mais aussi la performance, la sécurité, et la maintenabilité de l'application. Dans cette section, nous allons comparer quatre des frameworks les plus populaires : **Symfony**, **Laravel**, **Django** et **Spring Boot**, en examinant leurs caractéristiques, avantages et inconvénients.

II.1. Symfony

Caractéristiques :

Symfony est un framework open source pour PHP, qui utilise l'architecture MVC (Model-View-Controller) et est connu pour sa modularité. Il offre une bibliothèque riche de composants réutilisables qui permettent de construire des applications web sur mesure.

Avantages :

- **Modularité** : Grâce à sa structure modulaire, Symfony permet une personnalisation poussée. Les développeurs peuvent choisir les composants nécessaires.

- **Communauté Active** : Avec une vaste communauté autour, Symfony bénéficie d'un excellent support et de mises à jour régulières.
- **Documentation Complète** : La documentation détaillée facilite grandement l'apprentissage et la mise en œuvre du framework.

Inconvénients :

- **Courbe d'Apprentissage** : La richesse fonctionnelle et la complexité de Symfony peuvent en faire un choix intimidant pour les débutants.
- **Performances** : Bien que généralement performant, un projet mal configuré peut entraîner des lenteurs.

II.2. Laravel

Caractéristiques :

Laravel est également un framework PHP qui adopte l'architecture MVC. Il est célèbre pour sa syntaxe élégante et son approche axée sur l'expérience développeur.

Avantages :

- **Simplicité et Énergie** : Sa syntaxe intuitive permet d'écrire du code de manière claire, ce qui attire de nombreux développeurs.
- **Fonctionnalités Riches** : Avec des outils intégrés comme Eloquent ORM et des systèmes de routage, Laravel facilite le développement rapide d'applications.
- **Sécurité** : Laravel intègre des fonctionnalités de sécurité robustes par défaut, ce qui est essentiel pour protéger les applications contre des attaques courantes.

Inconvénients :

- **Performances** : Comme pour Symfony, une application trop chargée de fonctionnalités peut souffrir de problèmes de performance.
- **Dépendance à l'Écosystème** : Pour tirer le meilleur parti de Laravel, les développeurs doivent se plonger dans son écosystème, ce qui peut être contraignant.

II.3. Django

Caractéristiques :

Django est un framework Python qui suit également le schéma MVC (plus précisément le MVT - **Model-View-Template**). Il est réputé pour sa rapidité et sa capacité à simplifier le développement.

Avantages :

- **Développement Rapide** : Django est conçu pour faciliter le lancement rapide d'applications, grâce à des outils qui automatisent de nombreuses tâches courantes.
- **Sécurité Intégrée** : Le framework offre de nombreuses protections contre les vulnérabilités web, ce qui est un point fort pour les projets sensibles à la sécurité.

- **Documentation Exceptionnelle** : Sa documentation détaillée est un atout majeur pour les développeurs.

Inconvénients :

- **Rigidité** : Si Django impose des conventions, cela peut rendre difficile l'adoption d'approches différentes de celles prévues par le framework.
- **Poids** : Pour des applications simples, l'infrastructure de Django peut sembler surdimensionnée.

II.4. Sprint Boot

Caractéristiques :

Spring Boot est un framework Java qui simplifie le développement d'applications basées sur le framework Spring. Il est conçu pour faciliter la création de micro-services.

Avantages :

- **Écosystème Java** : Bénéficiant d'une vaste bibliothèque et d'un large éventail d'outils Java, Spring Boot permet une intégration facile avec d'autres technologies.
- **Développement Simplifié** : Avec des configurations par défaut et des conventions établies, Spring Boot permet un développement rapide et efficace.
- **Scalabilité** : Idéal pour les applications d'entreprise, Spring Boot est conçu pour être scalable, ce qui le rend adapté aux charges élevées.

Inconvénients :

- **Complexité Initiale** : Bien que Spring Boot simplifie la configuration par rapport à Spring, il nécessite une connaissance préalable de l'écosystème Spring.
- **Surcoût pour Petits Projets** : Pour des applications simples, il peut être excessif en termes de complexité et de poids.

II.5. Comparaison Globale

Critères	Symfony	Laravel	Django	Spring Boot
<i>Langage</i>	PHP	PHP	Python	Java
<i>Architecture</i>	MVC	MVC	MVT (Model-View-Template)	MVC
<i>Modularité</i>	Très Modulaire	Modulaire	Moins modulaire	Modulaire
<i>Performance</i>	Bonne, mais à optimiser	Bonne, mais à optimiser	Performance variable	Excellente, conçue pour cela
<i>Courbe d'Apprentissage</i>	Difficile	Facile	Moyenne	Difficile à moyenne
<i>Documentation</i>	Excellente	Excellente	Exceptionnelle	Très bonne
<i>Sécurité</i>	Solide	Solide	Excellente	Excellente

En somme, le choix entre Symfony, Laravel, Django et Spring Boot s'avère complexe et doit être réfléchi en fonction des exigences spécifiques du projet, des compétences de l'équipe et des objectifs à long terme. Chaque framework a ses propres forces et faiblesses, rendant leur pertinence variable selon le contexte d'utilisation.

Dans le cadre de ce cours, nous avons choisi d'utiliser Django en raison de plusieurs facteurs clés. Tout d'abord, Django facilite un développement rapide grâce à son approche intégrée qui automatise de nombreuses tâches courantes. Cela permet aux étudiants de se concentrer sur les concepts fondamentaux du développement web sans être submergés par des complexités techniques inutiles.

Ensuite, la sécurité est une priorité dans le développement d'applications web, et Django excelle dans ce domaine en offrant une multitude de protections intégrées contre les vulnérabilités les plus courantes. Cette caractéristique est particulièrement importante pour préparer les étudiants à concevoir des applications sécurisées dans un environnement professionnel.

De plus, la communauté active et la documentation approfondie de Django fournissent un soutien précieux aux étudiants. Cela facilite non seulement leur apprentissage, mais encourage également l'autonomie dans la résolution de problèmes courants.

Enfin, Django est un excellent choix pour introduire les étudiants à Python, un langage en forte demande sur le marché du travail. La maîtrise de Django et Python ouvre des portes vers diverses opportunités professionnelles dans divers domaines, allant du développement web à l'analyse de données et à l'IA.

Ainsi, en raison de ses fonctionnalités robustes, de son accent sur la sécurité et de sa capacité à simplifier le processus de développement, Django se révèle être le choix idéal pour ce cours. En utilisant Django, nous garantissons que les étudiants acquièrent des compétences précieuses qui les prépareront efficacement pour leurs futures carrières dans le domaine du développement web.

III. Installation et configuration de Python Django

Django est un framework web puissant et flexible qui permet de développer des applications web rapidement et efficacement en utilisant Python. Pour commencer à utiliser Django, il est essentiel de suivre des étapes d'installation et de configuration précises. Ce guide vous accompagnera à travers le processus d'installation de Django, que ce soit sur votre machine locale ou sur un environnement virtuel, ainsi que sur les bonnes pratiques de configuration.

Pré-requis :

Avant de commencer l'installation, assurez-vous que Python est installé sur votre système. Django nécessite une version de Python 3.6 ou supérieure. Pour vérifier si Python est installé, exécutez la commande suivante dans votre terminal ou invite de commande :

```
python --version
```

Ou

```
python3 --version
```

Si Python n'est pas installé, vous pouvez le télécharger depuis le [site officiel de Python](#).

Étape 1 : Installer pip

Pip est l'outil de gestion de paquets pour Python. Il vous permet d'installer et de gérer des bibliothèques Python, dont Django. Pour vérifier si pip est déjà installé, utilisez la commande suivante :

```
pip --version
```

Si pip n'est pas installé, vous pouvez l'installer en suivant les instructions disponibles sur le site officiel de pip : [Installation de pip](#).

Étape 2 : Créer un Environnement Virtuel

Il est recommandé d'utiliser un environnement virtuel pour isoler les dépendances spécifiques à chaque projet. Cela permet d'éviter les conflits de dépendances et de gérer différentes versions de bibliothèques.

Pour créer un environnement virtuel, utilisez les commandes suivantes :

1. Ouvrez votre terminal ou invite de commande.
2. Allez au répertoire de votre projet ou créez un nouveau répertoire :

```
pip --version
```

3. Créez un environnement virtuel :

```
python -m venv env
```

4. Activez l'environnement virtuel :

– Sur Windows :

```
env\Scripts\activate
```

– Sur macOS et Linux :

```
source env/bin/activate
```

Vous devriez voir le nom de votre environnement virtuel entre parenthèses au début de votre invite de commande, indiquant que l'environnement est actif.

Étape 3 : Installer Django

Avec l'environnement virtuel activé, vous pouvez installer Django à l'aide de pip. Pour installer la dernière version de Django, exécutez la commande suivante :

```
pip install django
```

Pour vérifier que l'installation a réussi, vous pouvez exécuter :

```
django-admin --version
```

Vous devriez voir le numéro de version de Django installé.

Étape 4 : Créer un Nouveau Projet Django

Une fois que Django est installé, vous pouvez créer un nouveau projet. Utilisez la commande suivante :

```
django-admin startproject nom_du_projet
```

Remplacez **nom_du_projet** par le nom que vous souhaitez donner à votre projet. Cette commande crée un répertoire portant le nom de votre projet, avec une structure de fichiers de base.

Étape 5 : Compréhension de la Structure de Dossier

À l'intérieur du répertoire de votre projet, vous trouverez les fichiers suivants :

- **manage.py** : Un script de commande pour interagir avec votre projet Django.
- **settings.py** : Contient les paramètres de configuration de votre projet.
- **urls.py** : Définit les URL de votre projet.
- **wsgi.py** : Connecte Django à un serveur web.
- **init.py** : Indique que le dossier doit être traité comme un paquet Python.

Étape 6 : Configurer la Base de Données

Django utilise SQLite par défaut pour la base de données, ce qui convient bien pour le développement. Cependant, vous pouvez le configurer pour utiliser d'autres bases de données comme PostgreSQL ou MySQL. Pour ce faire, modifiez le fichier **settings.py** :

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / "db.sqlite3", # Pour SQLite  
    }  
}
```

- Pour utiliser **PostgreSQL**, vous devez d'abord installer **psycopg2** :

```
pip install psycopg2
```

Ensuite, modifiez le **settings.py** :

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'nom_de_la_base_de_donnees',  
        'USER': 'nom_utilisateur',  
        'PASSWORD': 'mot_de_passe',  
        'HOST': 'localhost',  
        'PORT': '5432',  
    }  
}
```

Si vous rencontrez des problèmes lors de l'installation de **psycopg2**, vous pouvez essayer d'installer **psycopg2-binary**, une version précompilée.

```
pip install psycopg2-binary
```

- Pour utiliser **MySQL**, la procédure est légèrement différente :

Pour interagir avec MySQL, vous avez besoin d'un pilote spécifique. Installez **mysqlclient** en exécutant la commande suivante :

```
pip install mysqlclient
```

Notez que sur certaines plateformes, vous pourriez avoir besoin d'installer des dépendances supplémentaires (comme les en-têtes de développement MySQL) avant d'installer **mysqlclient**. Consultez la documentation officielle en cas de problème.

Ensuite, modifiez le **settings.py** :

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'nom_de_la_base_de_donnees',  
        'USER': 'nom_utilisateur',  
        'PASSWORD': 'mot_de_passe',  
        'HOST': 'localhost', # Changez cela si nécessaire  
        'PORT': '3306',      # Port standard pour MySQL  
    }  
}
```

Étape 7 : Créer les Tables de la Base de Données

Une fois que vous avez configuré la base de données, vous pouvez créer les tables correspondantes en exécutant les commandes suivantes :

```
python manage.py makemigrations  
python manage.py migrate
```

Cette commande appliquera toutes les migrations par défaut de Django et créera les tables nécessaires à la base de données.

Étape 8 : Démarrer le Serveur de Développement

Pour voir votre application en action, vous pouvez démarrer le serveur de développement intégré de Django en utilisant la commande suivante :

```
python manage.py runserver
```

Par défaut, le serveur sera accessible à l'adresse <http://127.0.0.1:8000/> dans votre navigateur.

Tests de connaissances

Questions à choix multiples (QCM)

1. Quel rôle principal joue un framework back-end ?
 - a. Styliser l'interface utilisateur
 - b. Gérer la logique métier, la persistance et les échanges serveur/client
 - c. Optimiser le référencement naturel
 - d. Compiler du code natif
2. Quelle architecture est la plus courante dans les frameworks back-end modernes ?
 - a. MVVM
 - b. MVP
 - c. MVC
 - d. Singleton
3. Lequel de ces frameworks est écrit en Python ?
 - a. Laravel
 - b. Symfony
 - c. Django
 - d. Spring Boot
4. Quel outil Python permet d'isoler les dépendances d'un projet Django ?
 - a. pipenv
 - b. venv
 - c. npm
 - d. composer
5. Quelle commande lance le serveur de développement Django ?
 - a. django start
 - b. python manage.py runserver
 - c. python server.py
 - d. npm run dev
6. Qu'est-ce qu'un framework back-end ?
 - a. Une bibliothèque pour créer des interfaces utilisateur.
 - b. Un ensemble d'outils et de normes facilitant le développement web côté serveur.
 - c. Un type de serveur web.
 - d. Un langage de programmation.
7. Quel est le rôle des trois composants de l'architecture MVC ?
 - a. Le Modèle gère les entrées utilisateur, la Vue gère la logique métier, le Contrôleur gère les données.
 - b. Le Modèle gère les données, la Vue présente les données, et le Contrôleur gère les entrées utilisateur.
 - c. Le Modèle présente les données, le Contrôleur gère les entrées utilisateur, et la Vue gère la base de données.
 - d. Le Contrôleur gère les données, la Vue gère la logique métier, et le Modèle gère la présentation.
8. Quel est un des inconvénients principaux des frameworks back-end ?

- a. Ils augmentent la productivité.
 - b. Ils améliorent la sécurité.
 - c. Ils peuvent ajouter une complexité inutile pour de petits projets.
 - d. Ils simplifient le développement.
9. Quel facteur est essentiel à considérer lors du choix d'un framework de développement ?
- a. La couleur de l'interface utilisateur du framework.
 - b. La popularité du framework sur les réseaux sociaux.
 - c. La scalabilité, la performance et la communauté de support.
 - d. La taille de la documentation.
10. Quels des frameworks suivants sont basés sur PHP ?
- a. Django et Spring Boot
 - b. Symfony et Laravel
 - c. Flask et Express
 - d. Rails et Vue
11. Quel est le fichier principal à modifier pour configurer la base de données dans un projet Django ?
- a. urls.py
 - b. manage.py
 - c. settings.py
 - d. wsgi.py

Questions à réponses ouvertes

1. Expliquez en quelques phrases pourquoi la modularité d'un framework est importante dans le développement d'applications.
2. Décrivez comment vous évalueriez les besoins d'un projet pour choisir un framework back-end approprié.
3. Donnez deux avantages et deux inconvénients majeurs de l'utilisation d'un framework back-end.
4. Expliquez brièvement le rôle du « Model » dans l'architecture MVC.
5. Pourquoi Django est-il choisi dans ce cours plutôt qu'un framework PHP ou Java ?
6. Citez trois critères à prendre en compte pour choisir un framework back-end.
7. Quelle est la différence entre pip et venv en Python ?
8. Un framework back-end se charge de la présentation visuelle de l'application. (Vrai / Faux)
9. Django utilise une architecture MVT (Model-View-Template). (Vrai / Faux)
10. Laravel est un framework Java. (Vrai / Faux)
11. La modularité est un avantage courant des frameworks back-end. (Vrai / Faux)
12. Il est impossible de migrer d'un framework à un autre une fois le projet lancé. (Vrai / Faux)

Etude de cas

1. Une Start-up qui veut une application en 3 semaines : justifiez le choix entre Django, Laravel et Spring Boot.
2. Comparez Symfony et Django pour une application de e-commerce sécurisée : 3 points forts et 3 risques pour chacun.